

Investigating the Limitations of Transformers with Simple Arithmetic Tasks

Alif Abdullah, Kevin Biliguun, Oliver Cheung, Vijay Krishnamoorthy

Introduction

LLMs are infamous for being unable to perform basic arithmetic, such as addition and subtraction. To investigate this dilemma, our original paper - titled “Investigating the Limitations of Transformers with Arithmetic Tasks” by Nogueira et al. - fine-tuned several pretrained T5 models on different numerical representations to demonstrate that different ways of displaying numbers can lead transformer models to achieve better performance.

Chosen Result

The specific result we chose to replicate is that 10-based (23 represented as “2 10 3”) and 10e-based (23 represented as “2 10e1 3”) numerical representations perform the best, while decimal numerical representations perform the worst across all chosen digit lengths. This is because these results show how transformers are poorly suited to doing arithmetic with our common way of displaying numbers. It also conveys two ways to effectively encode the true values of each of the digits in a number, achieving better mathematical results.

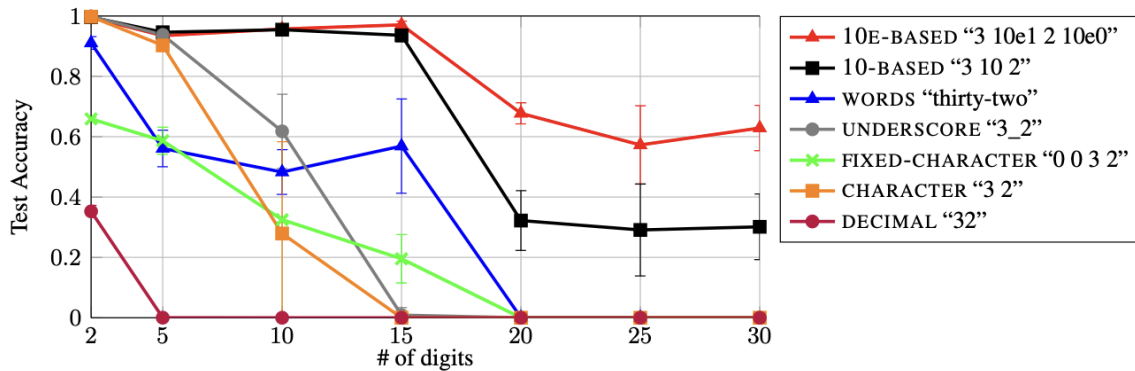


Figure 1: Accuracy of different number representations on the addition task.

Methodology

The original paper used the pretrained T5-small, T5-base, and T5-large models to perform their experimentation across different numerical representations. Furthermore, the original paper trained five sets of 1000 examples of digits for 100 epochs, using a sixth set of 1000 examples for evaluation. To keep compute costs down while also selecting models large enough to approximate the paper’s results, we went with fine-tuning the T5-base model. We also modified our dataset hyperparameters, using 20 epochs of 20000 samples for training, 200 samples for evaluation, and 1000 samples for testing.

Our inputs resemble the binary numerical operation template - “What is [number1] [operation] [number2]?”. While output samples follow the unary numerical template “[number3]”. Regardless, the paper ensures they are integers. Additionally, operation is either “plus” or “minus” for addition and subtraction, and the output is displayed in the same numerical representation as the inputs, which is the correct result of the input expression.

The paper listed two methods they used to generate data samples given a max digit length D . For training and evaluation, the paper used balanced sampling. Integer d is then sampled from $[2, D]$, at which point [number1] and [number2] are independently sampled from $[10^{d-1}, 10^d - 1]$, ensuring we have a balanced selection of length d numbers. For the test dataset, the paper used random sampling. Under this method, we independently sample [number1] and [number2] from $[0, 10^D - 1]$, resulting in 90% of our numbers having D digits, 9% having $D-1$ digits, 0.9% having $D-2$ digits, etc. We follow this exact dataset generation scheme when synthesizing our training, evaluation, and testing sets. However, due to compute costs, we limited our max number of digits to 15, while the original paper had 30. Regardless, the original paper evaluated results for 5, 10, 15, 20, 25, and 30, while our re-implementation evaluated results for 5, 10, and 15.

For the number representations, the original paper uses 7 representations, which are character “8 3 2”, fixed-character “0 8 3 2” (every number has the same number of max digits, using 0s for number places we do not use), underscore “8_3_2”, words “eight hundred thirty-two”, 10-based “8 100 3 10 2”, 10e-based “8 10e2 3 10e1 2”, and decimal “832”. We use the same numerical representations.

Along with these changes, we investigated the performance of our experimental procedure under several related tasks listed in the paper. Our extensions are subtraction, multiplication, and extrapolation. The first two are

self-explanatory, being subtraction and multiplication between two numbers. The third task involves training our models on some upper limit, then testing the model on number sequences with greater digit lengths than were used during training.

For all three additional tasks, we used the same dataset breakdown as before, used balanced sampling for training and evaluation sets, and random sampling for testing. However, the max digit length and numerical representation increments we evaluated differed. Only the subtraction task used all seven numerical representations we saw in the original paper. We would have liked to have done the same for multiplication and extrapolation, but since we were running out of credits on our Colab Pro plan, we stuck with 10-based and 10e-based numerical representations for multiplication and extrapolation. Additionally, we evaluated our approach - on the subtraction task - for D set to 5, 10, and 15. For multiplication, we attempted evaluation for D set to 3, 5, and 15 (skipping 10 as we saw terrible performance for 15, likely due to how big output sequences could get). For extrapolation, we trained our model on D set to 50, then evaluated our model on D set to 60.

Results & Analysis

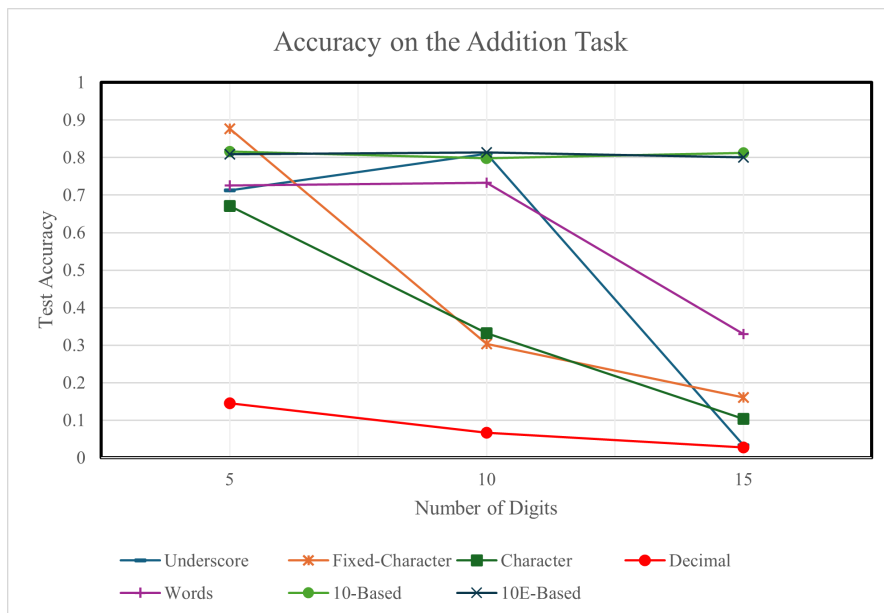


Figure 2: Our results for the Addition Task

Our results for the addition task show very similar trends to the paper's findings (see Figures 1 and 2). The 10-Based and 10E-Based representations were the most robust, having around 80% accuracy across all three digit lengths we tested. Decimal representation was consistently the worst-performing format, starting at 15% accuracy at digit length 5 before quickly dropping to near 0%. Underscore and Words performed well at 10 digits but had a sharp drop-off at 15 digits, while Fixed-Character and Character showed a steady decline across all digit lengths. These trends closely mirror the paper's results shown in Figure 1.

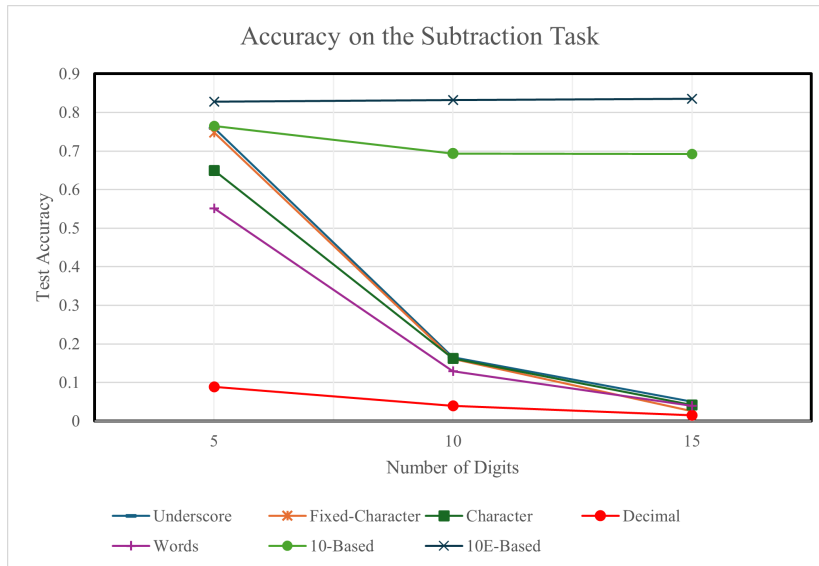


Figure 3: Our results for the Subtraction task

In Figure 3, we present our results for the subtraction task. Although the paper did not provide results for subtraction, the representations follow the same trends. Most formats other than Decimal performed well at 5 digits, but 10-Based and 10E-Based maintained strong accuracy (~70-85%) at 10 and 15 digits, while all other representations collapsed after 5 digits. This suggests that positional representations are even more important for subtraction than for addition.

Early in our re-implementation, 10-Based and 10E-Based were performing surprisingly poorly on addition, achieving only around 30% accuracy at 5 digits, while other formats performed much better. We discovered that this stemmed from a dataset generation issue: inputs were encoded in the target representation, but outputs were always in standard decimal. This forced the model to simultaneously learn the arithmetic task and the conversion from the representation back to decimal. Changing the output format to match the input greatly improved accuracy, bringing 10-Based and 10E-Based to around 90% at 5 digits. Notably, the paper's methodology section only describes how inputs were generated, and not outputs, so matching the input and output formats was an assumption we had to discover.

Our results reinforce the paper's finding that standard decimal notation is a poor choice for training transformers on arithmetic tasks. During tokenization, decimal representation obscures positional structure, whereas representations that explicitly encode magnitude provide local information about each digit's significance, allowing the model to learn digit position relationships more effectively. This suggests that tokenization choices have a significant impact on downstream task performance.

Reflections

Our re-implementation reinforced the paper's core idea about why certain representations perform better than others. Decimal suffers from inconsistent tokenization, making it difficult for the model to reconstruct numbers and learn digit significance. Character and Underscore build on this by introducing token boundaries, allowing for the model to infer digit significance by counting separators, but this task becomes increasingly difficult as digit length grows. Fixed-Character further helps by padding all numbers to the same length, giving the model a consistent positional reference, but also struggles with larger digit lengths. Word embeds positional significance through natural language structure (e.g., "thousand", "hundred"), but also fails to scale. 10-Based and 10E-Based solve this by encoding digit significance right next to each digit, making position a local operation. Our accuracy results across both addition and subtraction align well with these explanations.

Extensions We Explored

- **Extrapolation:** We trained 10-Based and 10E-Based models on 50 digit numbers and tested them on 60 digit addition. 10E-Based extrapolated well (66.6% accuracy) while 10-Based completely failed (0%). We suspect this is because 10-Based requires the model to generate large numbers of trailing zeros in longer numbers, while 10E-Based encodes this compactly through the exponent.
- **Hybrid Representation:** We combined 10E-Based with Underscore formatting for 15-digit addition, achieving 77.8% accuracy, slightly below the 80% of standard 10E-Based.

- Multiplication: Testing 10-Based and 10E-Based on multiplication at digit lengths 3, 5, 10, and 15 yielded near-zero accuracy across the board (1.5% at 3 digits, 0% otherwise). We attribute this to multiplication producing outputs with far more digits than the inputs, making it a significantly harder generalization task.

One extension we wanted to explore was trying out different bases (e.g., base 2 or 16). However, as the paper's related work notes, the T5 model during pre-training is not exposed to bases other than base 10. Therefore, fine-tuning alone would not be enough. We would have to include these different bases in the pretraining itself. Given compute constraints, this was not feasible, but we would be interested in seeing how the magnitude of the base affects arithmetic generalization.

References

Original Paper: "Investigating the Limitations of Transformers with Simple Arithmetic Tasks" Rodrigo Nogueira, Zhiying Jiang & Jimmy Lin

April 12, 2021

<https://arxiv.org/pdf/2102.13019>

T5 Model: 'Exploring the limits of transfer learning with a unified text-to-text transformer'

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, Peter J. Liu

Journal of Machine Learning Research, 2020

<https://arxiv.org/pdf/1910.10683>

Tools:

Python, Google Drive, Google Colab, PowerPoint, Google Docs

Libraries and Frameworks:

PyTorch, HuggingFace Transformers, HuggingFace Datasets, NumPy, Pandas, num2words